

z/OS Job Control Language (JCL)

YOUR FIRST JOB





FOUR NEW THINGS



You're ready to create your first job. To do that, you'll need to learn a little about four new things:

- Understanding what a "library" or "partitioned data set (PDS)" is
- Using the ISPF Editor
- The bare minimum JCL needed to run a job
- Using the System Display and Search Facility (SDSF)

LIBRARY OR PDS (PARTITIONED DATA SET)

On z/OS, a Library or PDS is physically a single file but logically a set of files that have the same attributes, such as logical record length, block size, and file organization. A PDS contains multiple data sets within it, called *members*, and a *directory* that keeps track of the relative locations of the members in the PDS.

The entire PDS can be treated as a single data set, but only for operations such as loading and unloading or compressing and decompressing the entire data set, without reference to the individual members.

Most of the time, individual members in a PDS are processed as if they were separate data sets.

If the name of the PDS is

USERID.MY.LIBRARY

then individual members in the Library could be referenced as

USERID.MY.LIBRARY (MEMBER1)

USERID.MY.LIBRARY (MEMBER2)

and so on.

LIBRARY OR PDS (PARTITIONED DATA SET)

We use PDSs or Libraries to organize our data sets in some reasonable way, much as we organize files under directories or folders on other systems.

Since all source has to be in the form of 80-byte text records that "look like" 80-column punched cards from the 1960s, we usually create PDSs to contain different kinds of source for different applications, for instance:

MYUSERID.COMMON.COBOL

MYUSERID.COMMON.COPY

MYUSERID.COMMON.JCL

MYUSERID.ASSET.MGMT.COBOL

MYUSERID.ASSET.MGMT.COPY

MYUSERID.ASSET.MGMT.JCL

MYUSERID.LOANORIG.COBOL

MYUSERID.LOANORIG.COPY

MYUSERID.LOANORIG.JCL

LIBRARY OR PDS (PARTITIONED DATA SET)

You've used ISPF to create a sequential dataset. Now let's see how to use the same ISPF facility to create a Partitioned Data Set, or Library. We'll create a Library to contain the JCL we write for lab exercises.

In ISPF we navigate to option 3.2 (Utilities → Data Set Utilities).

Then we enter "A" for "Allocate" in the Option field, and supply the High-Level Qualifier, Mid-Level Qualifier, and Data Set Name for the PDS we intend to create.

Then we press Enter.

```
Menu  RefList  Utilities  Help
-----
Data Set Utility

Option ==> a

A Allocate new data set
R Rename entire data set
D Delete entire data set
blank Data set information
C Catalog data set
U Uncatalog data set
S Short data set information
V VSAM Utilities

ISPF Library:
Project . . . ibmuser
Group . . . lab
Type . . . jcl

Enter "/" to select option
/ Confirm Data Set Delete

Other Partitioned, Sequential or VSAM Data Set:
Name . . . . .
Volume Serial . . . (If not cataloged, required for option "C")
Data Set Password . . (If password protected)
```

LIBRARY OR PDS (PARTITIONED DATA SET)

```

Menu  RefList  Utilities  Help
-----
Allocate New Data Set

Command ==>

Data Set Name . . . : IBMUSER.LAB.JCL

Management class . . . (Blank for default management class)
Storage class . . . SCBASE (Blank for default storage class)
Volume serial . . . USRVS1 (Blank for system default volume) **
Multiple Volumes . . . (Enter '/' to select option)
Device type . . . (Generic unit or device address) **
Data class . . . (Blank for default data class)
Space units . . . TRACK (BLKS, TRKS, CYLS, KB, MB, BYTES
                          or RECORDS)
Average record unit . . (M, K, or U)
Primary quantity . . 5 (In above units)
Secondary quantity . . 2 (In above units)
Directory blocks . . 0 (Zero for sequential data set) *
Record format . . . FB
Record length . . . 80
Block size . . . 27920
Data set name type . . LIBRARY (LIBRARY, PDS, LARGE, BASIC, *
                               EXTREQ, EXTPREF or blank)

```

The next panel echoes the data set name we provided, displaying it with the qualifiers separated by periods.

We enter other attributes we want the new data set to have. The key ones are near the bottom of the panel.

Record format FB means "fixed, blocked." The system will read/write *blocks*, and applications will read/write *logical records*.

The logical records are fixed length. The blocksize must be an even multiple of the logical record length.

Record length is 80 because punched cards had 80 columns.

The data set name type is LIBRARY, which will create a PDSE (PDS "extended") data set.

LIBRARY OR PDS (PARTITIONED DATA SET)

When we hit Enter, ISPF attempts to allocate the data set and returns us to the previous panel. It reports the result of the allocate operation in the upper right hand corner of the panel.

The values we entered for Project, Group, and Type are retained. If we press Enter now, without entering anything in the Option field, we'll see the data set information.

The screenshot shows the ISPF Data Set Utility panel. The title bar includes 'Menu', 'RefList', 'Utilities', and 'Help'. The main title is 'Data Set Utility'. In the top right corner, a red box highlights the text 'Data set allocated'. Below the title bar, there is a section for 'Option' with a red box around the 'Option ==>' label and a cursor. The main body of the panel lists several options: 'A Allocate new data set', 'R Rename entire data set', 'D Delete entire data set', 'blank Data set information', 'C Catalog data set', 'U Uncatalog data set', 'S Short data set information', and 'V VSAM Utilities'. Below this, there is a section for 'ISPF Library:' with a red box around it, containing fields for 'Project' (IBMUSER), 'Group' (LAB), and 'Type' (JCL). To the right of this section, there is a prompt 'Enter "/" to select option' and a confirmation message '/ Confirm Data Set Delete'. At the bottom, there is a section for 'Other Partitioned, Sequential or VSAM Data Set:' with fields for 'Name', 'Volume Serial', and 'Data Set Password', each with a corresponding prompt.

```
Menu  RefList  Utilities  Help

Data Set Utility

Option ==>

A Allocate new data set      C Catalog data set
R Rename entire data set    U Uncatalog data set
D Delete entire data set    S Short data set information
blank Data set information  V VSAM Utilities

ISPF Library:
Project . . . IBMUSER
Group . . . LAB
Type . . . JCL

Enter "/" to select option
/ Confirm Data Set Delete

Other Partitioned, Sequential or VSAM Data Set:
Name . . . . .
Volume Serial . . . (If not cataloged, required for option "C")
Data Set Password . . (If password protected)
```


LIBRARY OR PDS (PARTITIONED DATA SET)

```

Data Set Information
Command ==>

Data Set Name . . . . : IBMUSER.LAB.JCL

General Data
Management class . . . : **None**
Storage class . . . . : SCBASE
Volume serial . . . . : USRVS1
Device type . . . . . : 3390
Data class . . . . . : **None**
Organization . . . . . : PO
Record format . . . . : FB
Record length . . . . : 80
Block size . . . . . : 27920
1st extent tracks . . : 5
Secondary tracks . . . : 2
Data set name type : LIBRARY
Data set encryption : NO
Data set version . . : 1
SMS Compressible . . : NO

Current Allocation
Allocated tracks . . : 5
Allocated extents . . : 1
Maximum dir. blocks : NOLIMIT

Current Utilization
Used pages . . . . . : 5
% Utilized . . . . . : 8
Number of members . . : 0

Dates
Creation date . . . . : 2026/06/08
Referenced date . . . : ***None***
Expiration date . . . : ***None***
  
```

There's our data set name.

Organization PO means "partitioned organization," which is what we intended.

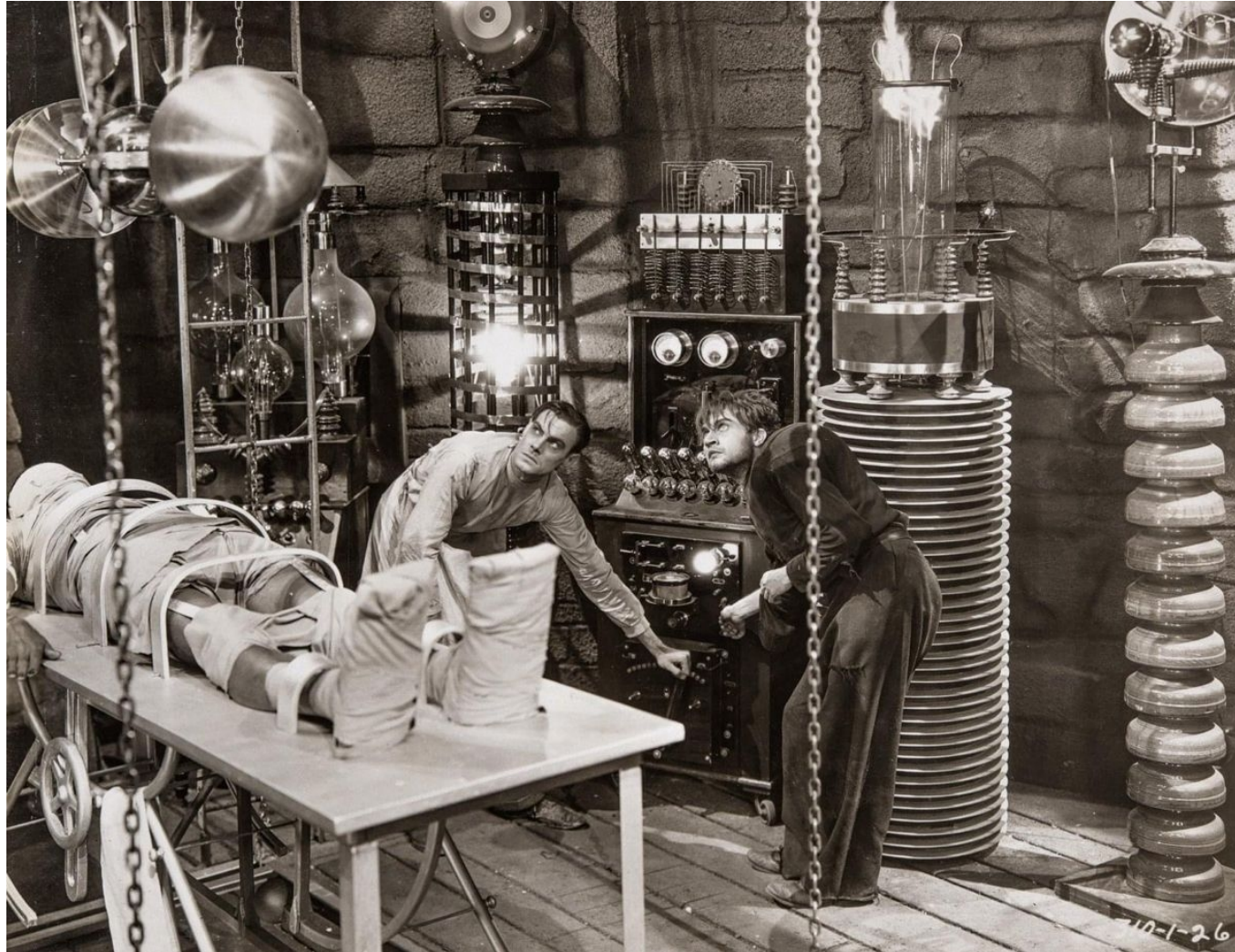
Maximum dir. Blocks NOLIMIT shows that the data set has a directory.

Data set name type LIBRARY shows we allocated a library.

All good.

A Library is a PDSE. There's an older format called PDS. To allocate a PDS, we specify a non-zero value for "directory blocks" and we specify "pds" as the "Data set name type". The old PDS format is supported for backward compatibility.

LAB Z/OS 7: CREATE A JCL LIBRARY USING ISPF



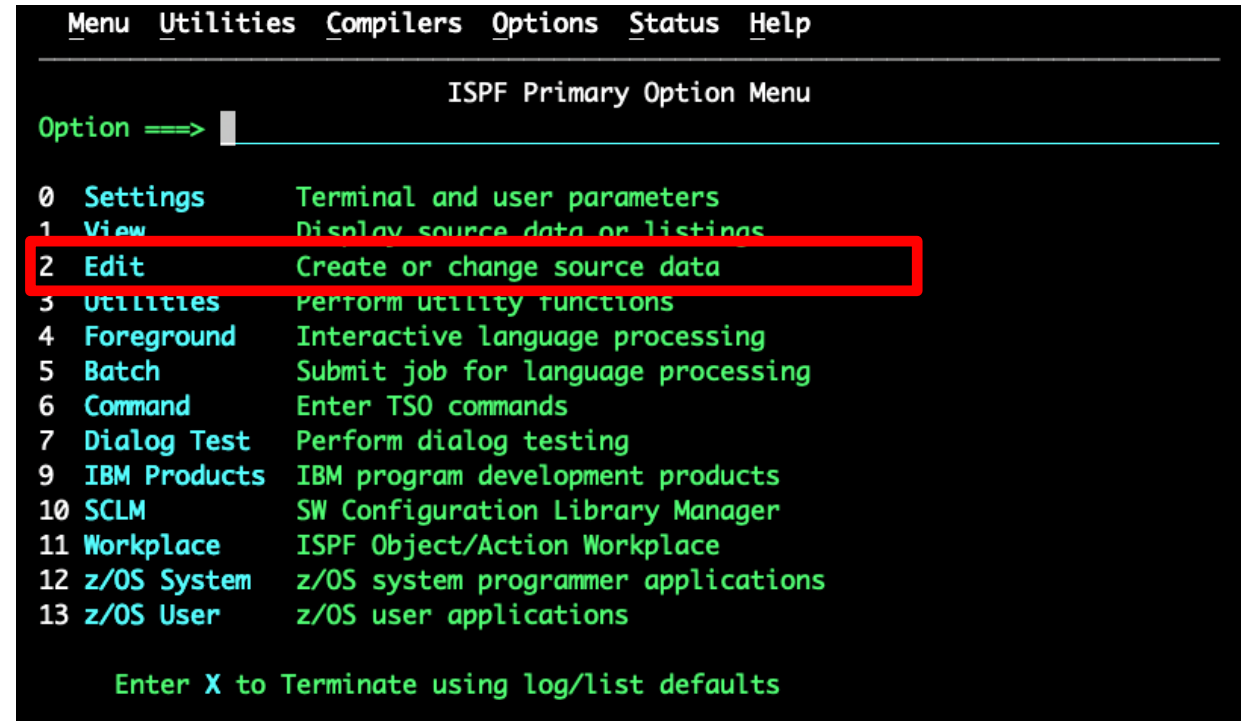
USING THE ISPF EDITOR

Now that you have a library for your JCL, the next step is to create the JCL for your first job. To do that, you need a text editor.

Unless you're using an off-platform front-end such as Zowe, the ISPF Editor will be the tool you use to edit data sets (files) on z/OS. It's a text-only editor of the same vintage as EMACS and VIM.

Like them, it is highly customizable, has its own way of doing things, and contains numerous cryptic commands and short-cuts. For someone new to the mainframe platform, learning the ISPF Editor is approximately as difficult as learning VIM, for someone new to the *nix world.

Our approach will be to introduce features of the editor as needed in the context of carrying out Lab assignments. As a Platform Engineer you will need to be familiar with the ISPF editor, but you need not become an expert with it. Access the editor via Option 2 on the ISPF Primary Option Menu.



USING THE ISPF EDITOR

```
Menu RefList RefMode Utilities Help
Edit Entry Panel
Command ==>
ISPF Library:
Project . . . ibmuser
Group . . . lab . . . . .
Type . . . jcl
Member . . . firstjob (Blank or pattern for member selection list)
Other Partitioned, Sequential or VSAM Data Set, or z/OS UNIX file:
Name . . . . . +
Volume Serial . . . (If not cataloged)
PDSE Generation . . . Options
Initial Macro . . . - Confirm Cancel/Move/Replace
Profile Name . . . - Mixed Mode
Format Name . . . - Preserve VB record length
Data Set Password . . Data Encoding
Record Length . . . - 1. ASCII
Line Command Table . . - 2. UTF-8
```

Option 2 on the ISPF Primary Option Menu takes us to the Edit Entry Panel. Here, we enter the name of the data set we want to edit.

We can type the name under "Other Partitioned", etc., or enter the HLQ, MLQ, and name under "ISPF Library" as shown at left.

The ISPF panels use the word Project for the HLQ, Group for the MLQ, and Type for the name, for historical reasons.

We've also entered the name of the *member* we want to edit.

IBMUSER.LAB.JCL is the name of the PDSE (Library) we just allocated. The system converts all these names to uppercase.

FIRSTJOB is the name of the member inside the Library where we're going to store the JCL for our first job.

The member doesn't exist. This is how you add a new member to a Library.

At this point we press Enter.

USING THE ISPF EDITOR

This is the Editor panel when you initially open a new data set or new member for editing. The blank lines are available for you to type into.

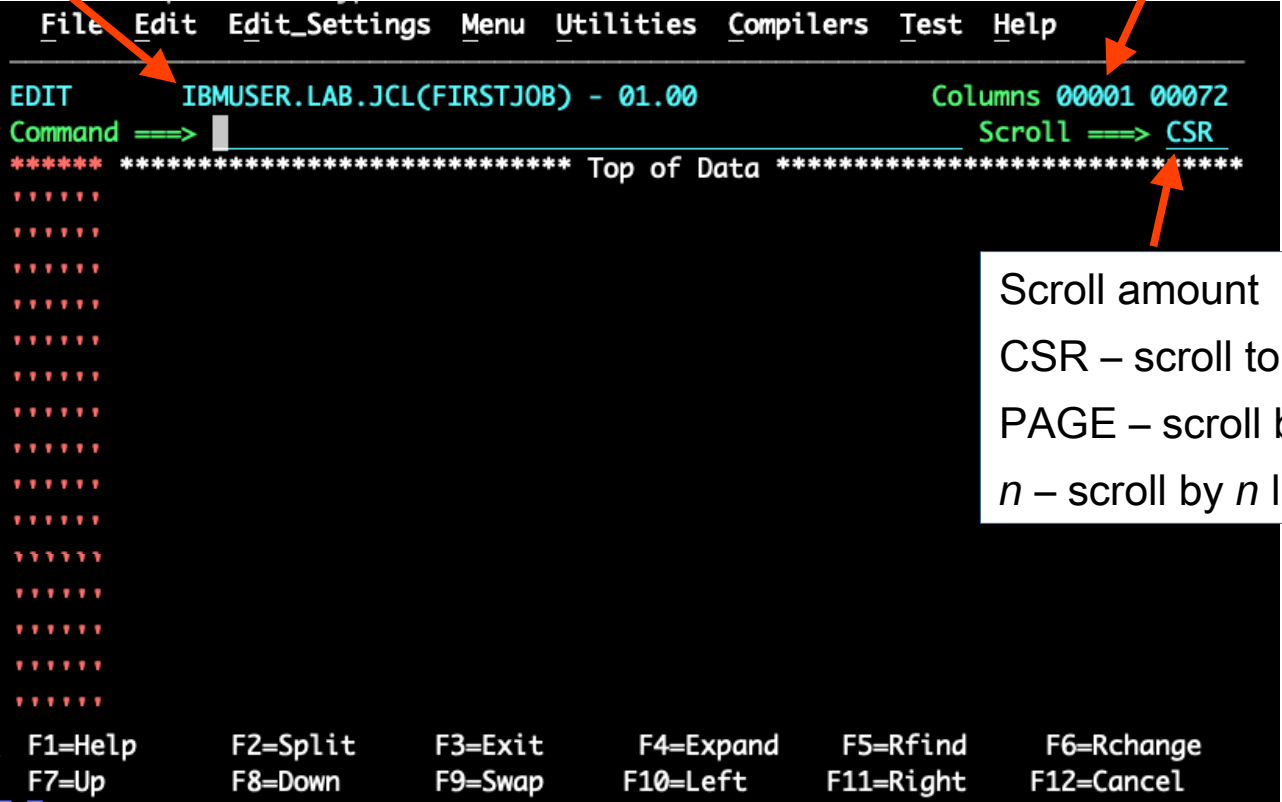
Name of data set (and member, if applicable) being edited.

Column numbers currently shown

Command field for Editor commands that apply to the whole file.

Line command entry fields

Function key help



Scroll amount
CSR – scroll to cursor position
PAGE – scroll by whole pages
n – scroll by *n* lines or columns

USING THE ISPF EDITOR

```
File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.00      Columns 00001 00072
Command ==>                                     Scroll ==> CSR
***** ***** Top of Data *****
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
v v v v v
```

This is a text-only environment and is designed to work with the keyboard and not with a pointing device. You use the Tab and Arrow keys to navigate around the panel.

You have to hit Tab four times to position the cursor at the beginning of the first line.

The cursor moves first into the Scroll field, then to the line of asterisks, then to the "Top of Data" line, and finally to the first "real" line of data, which is empty at this point.

YOUR JOB STATEMENT

The first statement in your JCL will be the JOB statement.

The JOB statement in all your jobs must identify you to JES2 so that it you can view the spooler output from the jobs.

Your userid is authorized to view only jobs that you own. If the value of the NOTIFY= parameter on the JOB statement is not the same as your TSO userid, you will never see the results of your job.

Use the &SYSUID symbol to ensure you will be notified and you will be able to view the spooler output.

```
//USERIDx  JOB    , 'YEAH' ,  
//          CLASS=A ,MSGCLASS=X ,MGSLEVEL=(1 ,1) ,  
//          NOTIFY=&SYSUID
```

This is all one single statement. It is continued on multiple lines. Many JCL statements have to be continued on multiple lines because originally they had to be punched on 80-column cards, and there weren't enough columns to contain the complete statement.

To continue a statement, type the comma after the last parameter on the line, type two slashes in positions 1 and 2 of the next line, leave at least one blank space after that, and resume typing the statement somewhere before column 16.

USING THE ISPF EDITOR

[illegible]

Here is how the panel looks after typing in the JOB statement into the first three blank lines. Use Tab to move to the next line after entering the data for each line.

When we press Enter, the remaining blank lines will go away. We can use *line commands* to open up more blank lines as needed.

USING THE ISPF EDITOR

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.00      Columns 00001 00072
Command ==>                                     Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1  JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
***** Bottom of Data *****

```

After pressing Enter, the panel looks like this.

The jobname starts with my userid, and has an additional character at the end; in this case, "1". We will be able to identify the job output in SDSF based on the jobname.

The keyword JOB is the next token in the JOB statement. One or more blank spaces follow JOB, and then the parameters are coded.

The first parameter is the job accounting information, which JES does not require but your organization may mandate a value here. We're not using it, so we have to indicate that it's omitted by coding a comma.

The second positional parameter is the programmer's name. Originally, this was helpful because all the output was printed, and we needed a way to identify who should receive the printout from the job. Today, this field usually contains a short text value that may suggest something about the purpose of the job. We're creating a job that does nothing, so I coded 'DO NOTHING' here.

The CLASS and MSGCLASS specifications are installation-dependent. For the labs, we use CLASS=A and MSGCLASS=X.

USING THE ISPF EDITOR

```
File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.00      Columns 00001 00072
Command ==>                                     Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1  JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
***** Bottom of Data *****
```

The NOTIFY keyword parameter tells the system which user to notify when the job starts and when it finishes. The system-defined symbol, &SYSUID, always contains the userid from which the job was submitted.

The first two parameters are positional and must be given in the right order. The remaining parameters are keyword parameters and can be coded in any order, after the positional parameters.

You can't code JCL statements beyond column 72. Columns 73 through 80 were used as identifiers to help re-sort the punched cards in the event the deck was dropped.

Of course, that is irrelevant today, but the Brain-in-a-Jar still thinks it's looking at punched cards, so we must adhere to the old rules.

SUBMITTING THE JOB

To run the job, we must submit the JCL to JES, the Job Entry Subsystem, for processing.

We enter the command SUBMIT in the Editor Command field. SUBMIT can be abbreviated as SUB. Press Enter.

TSO notifies us when the job starts, and also when it completes. In this case, the job runs so fast that we receive both notifications at once.

When TSO displays a message, it covers up the ISPF panel we were looking at. To dismiss the message, press Enter.

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.00      Columns 00001 00072
Command ==> sub                               Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
***** Bottom of Data *****

```

```

IKJ56250I JOB IBMUSER1(JOB01659) SUBMITTED
08.53.49 JOB01659 $HASP165 IBMUSER1 ENDED AT N1 - JCL ERROR CN(INTERNAL)
***

```

The completion message tells us the job ended with a JCL error. That means JES was unable to run the job because something was wrong with the JCL we submitted. We can check the job output using SDSF to see what happened.

SAVING THE NEW MEMBER

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.00      Columns 00001 00072
Command ==> save                               Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1  JOB ,'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
***** Bottom of Data *****

```

Before going to SDSF, we want to save our new member. We can do that either by typing the command SAVE in the Command field and pressing Enter, or by pressing the F3 key, which will save and exit.

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.00      Member FIRSTJOB saved
Command ==>                               Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1  JOB ,'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
***** Bottom of Data *****

```

ISPF tells us member FIRSTJOB was saved.

VIEWING SPOOLER OUTPUT

We press F3 repeatedly until we get back to the ISPF Primary Option Menu.

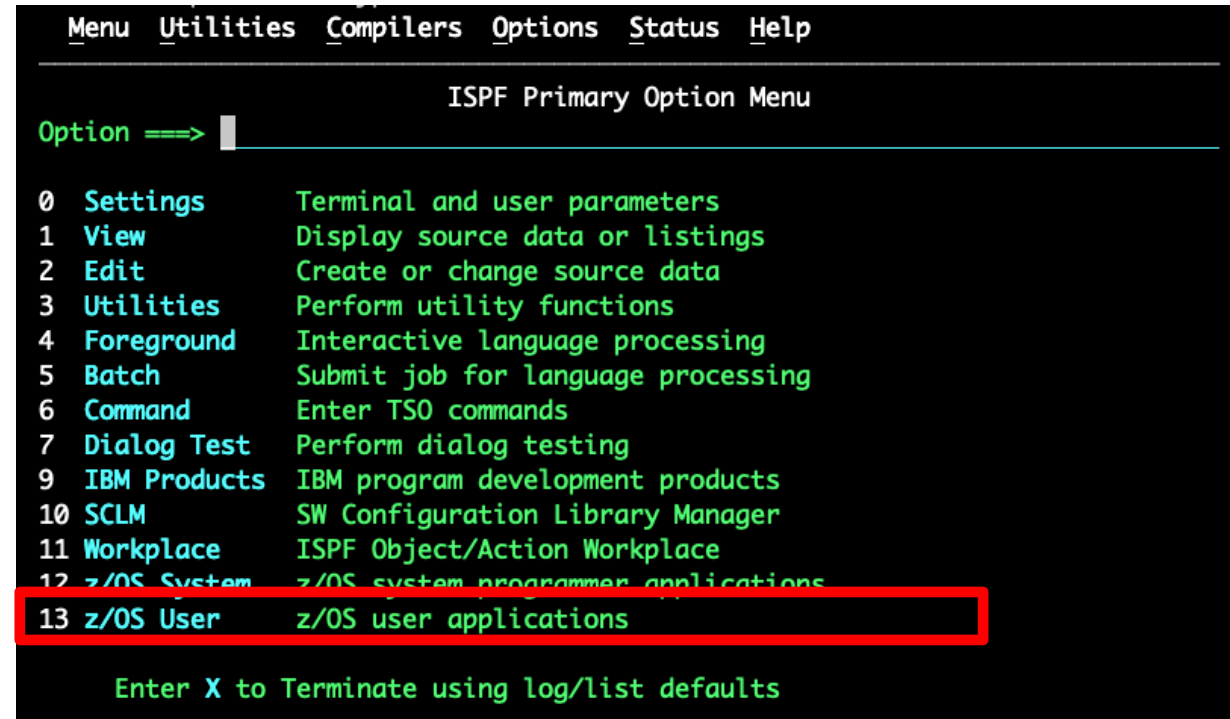
The option we're looking for now is SDSF, the System Display and Search Facility. This is the user interface for viewing job output that's managed by the system spooler.

Jobs can also write regular MVS data sets and USS files. Those are not managed by SDSF.

z/OS data sets that are conceptually equivalent to sysout and syserr on other systems are managed by SDSF.

By default, SDSF is Option S on the ISPF Primary Option Menu, but most organizations customize the ISPF menus beyond option 6 or 7. In our lab environment, they have put SDSF on a submenu. In your company, you'll have to learn where the SDSF option is located.

In our lab environment, we want to choose Option 13 on the Primary Option Menu.



VIEWING SPOOLER OUTPUT

```
z/OS Applications
Option ==> 
4 DFSMSrmm    DFSMSrmm/ISMF
5 DFSMSdftp   DFSMSdftp/ISMF
7 BDT         BDT File-to-File Feature
8 IPCS        IPCS for dump reading
9 Unix Browse z/OS Unix Browse Files
10 Unix Edit  z/OS Unix Edit Files
11 Unix Shell z/OS Unix Shell
12 Security   Security Server
13 TSO/E ICF  TSO/E Information Center Facility
14 SDSF       SDSF
15 OMVS       OMVS
S1 SPUFI      DB2 V12 SPUFI
S2 SPUFI      DB2 V13 SPUFI
MQ MQSeries   MQ for z/OS
```

We find the SDSF option on an installation-defined menu entitled z/OS Applications. It's option 14.

SDSF isn't a built-in part of ISPF. It's an ISPF application. That's the reason the systems programmers list it on a menu of applications instead of on a menu of ISPF functions.

VIEWING SPOOLER OUTPUT

SDSF displays a menu of choices. You will become familiar with most of these in the course of your work. For the purpose of viewing spooler output for our first job, we're interested in the Output Queue. That's menu option O.

When checking on the status of jobs we submit, we'll mainly look at options ST (Status), I (Input Queue), H (Held Queue), and O (Output Queue).

Perusing the list of options, you can see that a lot of information about the system is available through this facility. We'll check that out later. For now, we want to see what happened with our first job.

We type O in the Command Input field and press enter.

```

Display Filter View Print Options Search Help
-----
SDSF MENU 3.1 LOCAL VS01 LINE 1-38 (98)
COMMAND INPUT ==> |
NP NAME Description Group Status
DA Active users Jobs
T Input queue Jobs
O Output queue Output
H Held queue Jobs
ST Status of jobs Jobs
JG Job groups JES
SYM System symbols System
LOG System log Log
SR System requests Log
MAS Members in the MAS JES
JC Job classes JES
SE Scheduling environments WLM
RES WLM resources WLM
ENC Enclaves WLM
PS Processes OMVS
SYS System information System
ENQ Enqueues System
ENQC Enqueue contention System
ENQD Enqueued data sets Sysplex
DYNX Dynamic exits System
AS Address space memory Jobs
INIT Initiators JES
PR Printers JES
PUN Punches JES
RDR Readers JES
LINE Lines Network
NODE Nodes Network
SO Spool offload JES
SP Spool volumes JES
NS Network servers Network
NC Network connections Network
RM Resource monitor JES
CK Health checker System
LNK Link list data sets System
LPA Link pack data sets System
APF APF data sets System
F1=HELP F2=SPLIT F3=END F4=RETURN F5=RFIND F6=RCHANGE
F7=UP F8=DOWN F9=SWAP F10=LEFT F11=RIGHT F12=RETRIEVE
  
```

VIEWING SPOOLER OUTPUT

Display Filter View Print Options Search Help									

SDSF OUTPUT ALL CLASSES ALL FORMS					LINES 435		LINE 1-8 (8)		
COMMAND INPUT ==>							SCROLL ==> CSR		
NP	JOBNAME	JobID	Owner	Prty C	Forms	Dest	Tot-Rec		
	IBMUSER	TSU01540	IBMUSER	144 A	STD	LOCAL	34		
	IBMUSER	TSU01599	IBMUSER	144 A	STD	LOCAL	19		
	IBMUSER	TSU01606	IBMUSER	144 A	STD	LOCAL	19		
	IBMUSER	TSU01607	IBMUSER	144 A	STD	LOCAL	19		
	IBMUSER	TSU01610	IBMUSER	144 A	STD	LOCAL	253		
	IBMUSERN	JOB01614	IBMUSER	144 X	STD	LOCAL	41		
	IBMUSERN	JOB01615	IBMUSER	144 X	STD	LOCAL	33		
	IBMUSER1	JOB01659	IBMUSER	144 X	STD	LOCAL	17		

The SDSF Output Queue panel is filtered to show only job output associated with my userid, IBMUSER.

What you see listed includes previous TSO sessions, the current TSO session, and output from three batch jobs that I ran recently.

The "first job" job that we're looking for is there at the end of the list, with jobname IBMUSER1.

To view the data sets for job IBMUSER1, we type a question mark (?) in the field to the left of the line of data. There's no visual indication a field is present there, but it's there.

The question mark will let us see the individual output data sets separately. We could also type S in that field to see all the output data sets concatenated.

Display Filter View Print Options Search Help									

SDSF OUTPUT ALL CLASSES ALL FORMS					LINES 435		LINE 1-8 (8)		
COMMAND INPUT ==>							SCROLL ==> CSR		
NP	JOBNAME	JobID	Owner	Prty C	Forms	Dest	Tot-Rec		
	IBMUSER	TSU01540	IBMUSER	144 A	STD	LOCAL	34		
	IBMUSER	TSU01599	IBMUSER	144 A	STD	LOCAL	19		
	IBMUSER	TSU01606	IBMUSER	144 A	STD	LOCAL	19		
	IBMUSER	TSU01607	IBMUSER	144 A	STD	LOCAL	19		
	IBMUSER	TSU01610	IBMUSER	144 A	STD	LOCAL	253		
	IBMUSERN	JOB01614	IBMUSER	144 X	STD	LOCAL	41		
	IBMUSERN	JOB01615	IBMUSER	144 X	STD	LOCAL	33		
?	IBMUSER1	JOB01659	IBMUSER	144 X	STD	LOCAL	17		

VIEWING SPOOLER OUTPUT

Display Filter View Print Options Search Help									

SDSF JOB DATA SET DISPLAY - JOB IBMUSER1 (JOB01659)					LINE 1-3 (3)				
COMMAND INPUT ==> █					SCROLL ==> CSR				
NP	DDNAME	StepName	ProcStep	DSID	Owner	C	Dest	Rec-Cnt	Page
	JESMSGLG	JES2		2	IBMUSER	X	LOCAL	11	
	JESJCL	JES2		3	IBMUSER	X	LOCAL	4	
	JESYSMSG	JES2		4	IBMUSER	X	LOCAL	2	

We've finally drilled down to the point that we can see the output files produced by JES for our job.

There are many details to learn and we haven't gone through a comprehensive introduction about JES or SDSF. We're picking up details little by little in the context of running our first job.

Your knowledge will build as you keep doing things on the system. Don't worry.

The three data sets you see here are standard. JES writes them for every job it processes.

The data sets can be suppressed by values in the MSGLEVEL= parameter of the JOB statement. For production jobs we don't need to see all this output unless there's a serious error. For learning purposes we want to see all the output that's produced.

JESMSGLG contains messages about JES job processing.

JESJCL contains the JCL with symbols resolved to actual values, and with the contents of JCL Procedures (PROCs) expanded, so we can see the exact JCL JES was working with.

JESYSMSG contains messages about what happened during job execution. Most useful error messages will be found here.

We view one of these data sets by typing an S to the left of it and pressing Enter.

JESMSGGLG DATA SET

This is the JESMSGGLG output for our job.

It tells us the job is assigned to userid IBMUSER.

It tells us the job was not executed because there was an error in the JCL, and it provides error code 446, which we can look up in the documentation.

It reports some basic statistics about the job. You can see more throwbacks to the Olden Days here, such as references to cards and to print and punch output record counts.

The job ran – or failed to run – in no time at all.

We exit from this with F3, and we can choose the next spooler data set to view.

```

Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01659 DSID      2 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> █      SCROLL ==> CSR
***** TOP OF DATA *****
                J E S 2  J O B  L O G  --  S Y S T E M  V S 0 1  --  N

08.53.48 JOB01659 ---- TUESDAY,  09 JUN 2026 ----
08.53.48 JOB01659 IRR010I  USERID IBMUSER  IS ASSIGNED TO THIS JOB.
08.53.48 JOB01659 IEFC452I  IBMUSER1 - JOB NOT RUN - JCL ERROR  446
----- JES2 JOB STATISTICS -----
                3 CARDS READ
                18 SYSOUT PRINT RECORDS
                 0 SYSOUT PUNCH RECORDS
                 1 SYSOUT SPOOL KBYTES
                0.00 MINUTES EXECUTION TIME
***** BOTTOM OF DATA *****

```


JESJCL DATA SET

```

  Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01659 DSID      3 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> █      SCROLL ==> CSR
***** TOP OF DATA *****
  1 //IBMUSER1  JOB , 'DO NOTHING',
    //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
    //          NOTIFY=&SYSUID
    IEFC653I SUBSTITUTION JCL - , 'DO NOTHING',CLASS=A,MSGCLASS=X,MSGLEVEL=
***** BOTTOM OF DATA *****

```

This is the JESJCL output for our job.

It shows us the exact JCL that JES processed, including expanded PROCs and resolved symbols. In our case, the JCL is pretty simple and there isn't much to show beyond what we can see in the Editor.

We have to scroll to the right to see all the data. The initial display (above) shows columns 2 through 81, and the scrolled display (right) shows columns 54 through 133.

Spooler output data sets have 133 byte records. This reflects the line length of printers in the Olden Days, which had 132 bytes of text plus one byte for carriage control codes. The Brain-in-a-Jar thinks it's still working with printer output.

No PROCs are referenced in this job. The only symbol was in the NOTIFY=&SYSUID parameter, and we see that the value IBMUSER was substituted.

```

  Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01659 DSID      3 LINE 0      COLUMNS 54- 133
COMMAND INPUT ==> █      SCROLL ==> CSR
***** TOP OF DATA *****
                                JOB01659
=(1,1),
CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),NOTIFY=IBMUSER
***** BOTTOM OF DATA *****

```

JESYSMSG DATA SET

This is the JESYSMSG output for our job.

This data set can be quite long. It contains information about everything that happened during processing of the job. Every step that ran, the condition codes or abend codes with which it ended, every data set that was allocated or deleted or discarded or kept.

```

Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01659 DSID 4 LINE 0 COLUMNS 02- 81
COMMAND INPUT ==> | SCROLL ==> CSR
***** TOP OF DATA *****
STMT NO. MESSAGE
      1 IEF607I JOB HAS NO STEPS
***** BOTTOM OF DATA *****

```

For our job, JESYSMSG doesn't have a whole lot to say, but what it says tells us a lot.

JOB HAS NO STEPS tells us that we can't have a job that consists solely of a JOB statement, even if our intent is that the job doesn't do anything.

You will recall that the JCL EXEC statement defines a job step. Our job doesn't have one. We need to add an EXEC statement to our JCL. But what will the EXEC statement execute, if we want the job to do nothing?

To relate this to Unix/Linux, think about the program named *true*. It sets the status code to zero and exits. Microsoft Windows® doesn't have this, but it's easy enough to write a cmd script or PowerShell script that exits with zero.

z/OS has a program like this, named IEFBR14. There's a potentially-interesting back story about that name which we'll skip for now. Suffice it to say IEFBR14 sets the condition code to zero and exits.

The usefulness of this on z/OS is that JES itself can do a lot of things with data sets based on the JCL DD statements in a job step, without the need for a user-written program. But it needs to have a program to run rather than no program at all.

PURGING SPOOLER OUTPUT

Display Filter View Print Options Search Help							

SDSF OUTPUT ALL CLASSES ALL FORMS				LINES 435		LINE 1-8 (8)	
COMMAND INPUT ==>						SCROLL ==> CSR	
NP	JOBNAME	JobID	Owner	Prty	C	Forms	Dest Tot-Rec
	IBMUSER	TSU01540	IBMUSER	144	A	STD	LOCAL 34
	IBMUSER	TSU01599	IBMUSER	144	A	STD	LOCAL 19
	IBMUSER	TSU01606	IBMUSER	144	A	STD	LOCAL 19
	IBMUSER	TSU01607	IBMUSER	144	A	STD	LOCAL 19
	IBMUSER	TSU01610	IBMUSER	144	A	STD	LOCAL 253
p	IBMUSERN	JOB01614	IBMUSER	144	X	STD	LOCAL 41
p	IBMUSERN	JOB01615	IBMUSER	144	X	STD	LOCAL 33
p	IBMUSER1	JOB01659	IBMUSER	144	X	STD	LOCAL 17

Before we return to the Editor to add an EXEC statement to our job, let's get rid of the spooler output. It's rare that we need to keep spooler output very long. In this case, we've gotten the information we need from it and there's no need to keep it around. If we let spooler data sets accumulate, we'll soon lose track of which job was which.

Most z/OS installations are configured to purge spooler output after a day or so, but it's still a good idea to clear out things we don't need anymore.

To purge spooler data sets, type a P next to each job in the queue. SDSF can process an arbitrary number of P commands in one go. After pressing Enter, one or more of the purged jobs will still be visible on the SDSF display. It takes a couple of seconds to clear completely.

Notice that I didn't purge my TSO session. That wouldn't destroy anything, but I'd have to sign on again.

```

Menu  RefList  RefMode  Utilities  Help
-----
Edit Entry Panel

Command ==> _____

ISPF Library:
Project . . . IBMUSER
Group . . . LAB . . . _____ . . . _____
Type . . . JCL
Member . . . _____ (Blank or pattern for member selection list)

Other Partitioned, Sequential or VSAM Data Set, or z/OS UNIX file:
Name . . . . . _____ +
Volume Serial . . _____ (If not cataloged)

Options
PDSE Generation . . . _____ - Confirm Cancel/Move/Re
Initial Macro . . . _____ - Mixed Mode
Profile Name . . . _____ - Preserve VB record len
Format Name . . . _____
Data Set Password . . _____
Record Length . . . _____
Line Command Table . . _____

Data Encoding
- 1. ASCII
- 2. UTF-8

```

We can select it by typing an S next to its name on the list.

```

Menu  Functions  Utilities  Help
-----
EDIT      IBMUSER.LAB.JCL                      Row 0000001 of 0000001
Command ==> |                                     Scroll ==> PAGE
      Name      Prompt      Size      Created      Changed      ID
. FIRSTJOB      3      2026/06/09      2026/06/09 08:58:00      IBMUSER
**End**

```

ADDING AN EXEC STATEMENT TO OUR JCL

To insert a line into our FIRSTJOB member, we type the line command i (for insert) on line 3 and press Enter.

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.02          Columns 00001 00072
Command ==>                                         Scroll ==> CSR
***** ***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
i 0003 //          NOTIFY=&SYSUID
***** ***** Bottom of Data *****

```

The insert line command opens up a blank line just after the line where we typed it.

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.02          Columns 00001 00072
Command ==>                                         Scroll ==> CSR
***** ***** Top of Data *****
000001 //IBMUSER1 JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
***** ***** Bottom of Data *****

```

ADDING AN EXEC STATEMENT TO OUR JCL

```

File Edit Edit_Settings Menu Utilities Compilers Test Help
EDIT      IBMUSER.LAB.JCL(FIRSTJOB) - 01.01      Columns 00001 00072
Command ==>                                     Scroll ==> CSR
***** Top of Data *****
000001 //IBMUSER1  JOB , 'DO NOTHING',
000002 //          CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
000003 //          NOTIFY=&SYSUID
000004 //NADA      EXEC PGM=IEFBR14
***** Bottom of Data *****

```

```

10.20.30 JOB01670 $HASP165 IBMUSER1 ENDED AT N1 MAXCC=0000 CN(INTERNAL)
***

```

Here's the Editor panel after we added an EXEC statement specifying program IEFBR14.

After we submit the job, TSO notifies us that the job completed with a maximum condition code of zero.

In z/OS, that means no errors were detected by the system. It doesn't necessarily mean the result of the job was what we wanted or expected. We still need to check the output to see what happened.

SDSF OUTPUT

DisplayFilterViewPrintOptionsSearchHelp

SDSF OUTPUT ALL CLASSES ALL FORMS LINES 378 LINE 1-6 (6)

COMMAND INPUT ==> SCROLL ==> CSR

NP	JOBNAME	JobID	Owner	Prty	C	Forms	Dest	Tot-Rec
	IBMUSER	TSU01540	IBMUSER	144	A	STD	LOCAL	34
	IBMUSER	TSU01599	IBMUSER	144	A	STD	LOCAL	19
	IBMUSER	TSU01606	IBMUSER	144	A	STD	LOCAL	19
	IBMUSER	TSU01607	IBMUSER	144	A	STD	LOCAL	19
	IBMUSER	TSU01610	IBMUSER	144	A	STD	LOCAL	253
?	IBMUSER1	JOB01670	IBMUSER	144	X	STD	LOCAL	34

Going to SDSF, we choose our job from the Output Queue and use "?" to see the spooler data sets.

Now we'll take a look at JESMSGLG and JESYSMSG.

DisplayFilterViewPrintOptionsSearchHelp

SDSF JOB DATA SET DISPLAY - JOB IBMUSER1 (JOB01670) LINE 1-3 (3)

COMMAND INPUT ==> SCROLL ==> CSR

NP	DDNAME	StepName	ProcStep	DSID	Owner	C	Dest	Rec-Cnt	Page
	JESMSGLG	JES2		2	IBMUSER	X	LOCAL	16	
	JESJCL	JES2		3	IBMUSER	X	LOCAL	5	
	JESYSMSG	JES2		4	IBMUSER	X	LOCAL	13	

SDSF OUTPUT

```
  _Display_ _Filter_ _View_ _Print_ _Options_ _Search_ _Help_
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01670  DSID      2 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> |      SCROLL ==> CSR
***** TOP OF DATA *****
      J E S 2  J O B  L O G  --  S Y S T E M  V S 0 1  --  N

10.20.30 JOB01670 ---- TUESDAY,   09 JUN 2026 ----
10.20.30 JOB01670 IRR010I  USERID IBMUSER  IS ASSIGNED TO THIS JOB.
10.20.30 JOB01670 ICH70001I IBMUSER  LAST ACCESS AT 06:26:49 ON TUESDAY, JUNE 9
10.20.30 JOB01670 $HASP373 IBMUSER1  STARTED - INIT 1    - CLASS A      - SYS
10.20.30 JOB01670 IEF403I  IBMUSER1  - STARTED - TIME=10.20.30
10.20.30 JOB01670 IEF404I  IBMUSER1  - ENDED  - TIME=10.20.30
10.20.30 JOB01670 $HASP395 IBMUSER1  ENDED  - RC=0000
----- JES2 JOB STATISTICS -----
      09 JUN 2026 JOB EXECUTION DATE
           4 CARDS READ
          38 SYSOUT PRINT RECORDS
           0 SYSOUT PUNCH RECORDS
           6 SYSOUT SPOOL KBYTES
          0.00 MINUTES EXECUTION TIME
***** BOTTOM OF DATA *****
```

JESMSGGLG reports the job ran with condition code zero. No errors.

SDSF OUTPUT

JESYSMSG is a lot more interesting than it was on our first attempt.

The "job related settings" are installation defaults; we didn't override any of that.

Step NADA was executed and completed with condition code zero.

We see the start and end times for step NADA.

We see CPU usage and memory allocation.

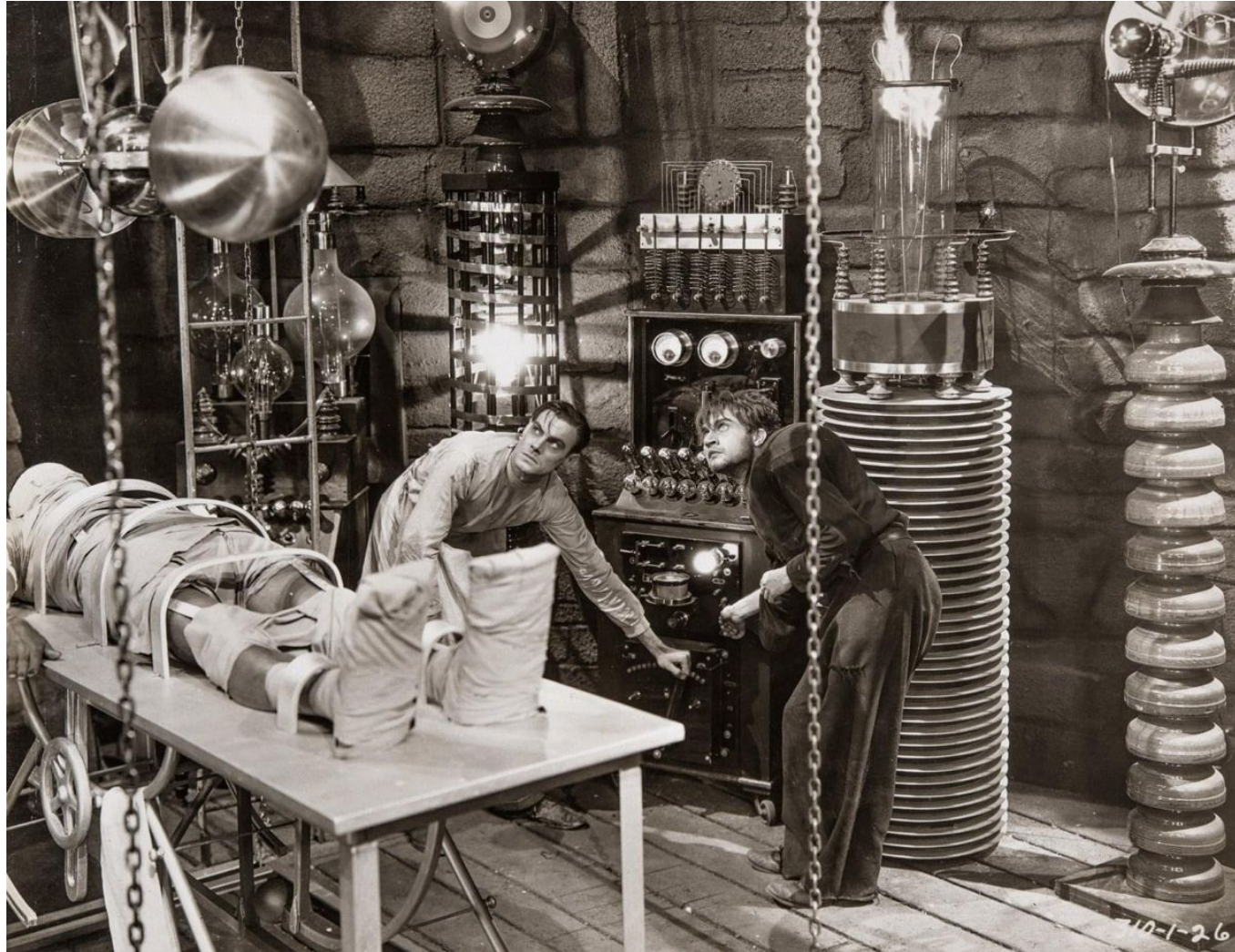
We see no scary error messages.

```

Display Filter View Print Options Search Help
-----
SDSF OUTPUT DISPLAY IBMUSER1 JOB01670 DSID      4 LINE 0      COLUMNS 02- 81
COMMAND INPUT ==> █ SCROLL ==> CSR
***** TOP OF DATA *****
ICH70001I IBMUSER  LAST ACCESS AT 06:26:49 ON TUESDAY, JUNE 9, 2026
IEFA111I IBMUSER1 IS USING THE FOLLOWING JOB RELATED SETTINGS:
        SWA=BELOW,TIOT SIZE=64K,DSENQSHR=DISALLOW,GDGBIAS=JOB
IEF142I IBMUSER1 NADA - STEP WAS EXECUTED - COND CODE 0000
IEF373I STEP/NADA      /START 2026160.1020
IEF032I STEP/NADA      /STOP  2026160.1020
        CPU:      0 HR  00 MIN  00.00 SEC      SRB:      0 HR  00 MIN  00.00 SEC
        VIRT:      8K  SYS:   288K  EXT:      0K  SYS:   9868K
        ATB- REAL:      1248K  SLOTS:      0K
        VIRT- ALLOC:      14M  SHRD:      0M
IEF375I  JOB/IBMUER1/START 2026160.1020
IEF033I  JOB/IBMUER1/STOP  2026160.1020
        CPU:      0 HR  00 MIN  00.00 SEC      SRB:      0 HR  00 MIN  00.00 SEC
***** BOTTOM OF DATA *****

```

LAB Z/OS 8: CREATE AND RUN A DO-NOTHING JOB



WHAT YOU KNOW AND WHAT YOU CAN DO

- You can code a bare minimum job that runs.
- You know what you must include on the JOB statement to ensure you will be able to find and view the job's output.
- You can locate the spooler output from a job and review it.

WHAT'S NEXT

- More about data sets and files on z/OS